

JS Executor

Библиотека JS Executor необходима для того, чтобы предоставить нашим клиентам возможность создать свою логику, которую изначально мы не закладывали в систему, например, при создании собственных печатных форм в документообороте компании. Иными словами, пользователь на базе данных из наших бизнес-объектов получает данные, обрабатывает их по своему усмотрению и выводит на экран собственную печатную форму.

Термины и определения

Контекст (задача) - область применения исполнителя JS кода. У контекста есть набор параметров и возможно есть функций расширения. Примеры: проведение документов, расчет зарплаты.

Функция расширения - функция, написанная на Python, реализующая некоторый функционал. Для функции расширения указывается имя, по которому ее можно вызывать из JS.

Операция - код, который вызывает прикладной программист.

Функция - вспомогательный код клиента.

Архитектура системы

Для выполнения JS кода мы используем Google V8. Для совместимости с платформой написан модуль на C++, а также несколько вспомогательных классов на Python. Весь код объединен в модуль JS Executor в репозитории [executor](#). Модуль можно использовать только из Python.

Процесс внедрения

Создание контекста



Помимо создания контекста в коде на Python, необходимо зарегистрировать код в [Сервисе прикладного кода](#).

Для контекста нужно определить уникальный идентификатор. Он нужен для добавления функций расширения, пользовательских функций на JS, а также для получения объекта исполнителя JS-кода.

Контекст нужно зарегистрировать при старте сервиса для дальнейшего использования. Регистрация выполняется с помощью файла `on_event.py`.

Это условие необязательное, главное, чтобы соблюдались порядок регистрации контекста, регистрация функций расширения и только потом использование.

```
1 from ExecutorExtender import ExecutorExtender
2 MY_CONTEXT_UNIQUE_IDENTIFIER = '...'
3 ExecutorExtender.registerTask( MY_CONTEXT_UNIQUE_IDENTIFIER,
  ExecutionTimeout )
```

Параметры контекста

Все параметры объединяем в python-класс, который наследуется от `BaseTaskParams`. Объект этого класса с установленными параметрами передается в функцию, исполняющую JS. Также объект передается первым параметром в функции расширения.

Например, для контекста Расчет зарплаты:

```
1 class ZplCalcTaskParams( BaseTaskParams ):
2     # Зарплату мы считаем для сотрудника будем хранить его первичный
  ключ
```

```

3     manId = None
4     # в определенном месяце
5     month = None
6     # и т.д.

```

Функции расширения

Функция расширения - это функция на Python. Первым параметром всегда приходит объект класса с параметрами контекста, остальные параметры передаются из JS. Можно передавать только позиционные параметры, именованные параметры не поддерживаются в JS. Но можно эмулировать это, передавая JS-object, который преобразуется в словарь Python dict.

Можно также указывать значения по умолчанию для параметров и использовать позиционные аргументы *args.

Для того чтобы JS Executor связать с функцией, необходимо добавить в нее декоратор jsExtension и импортировать файл с функцией после регистрации контекста, но до исполнения JS-кода. Эта процедура выполняется в файле on_event.py.

Имя функции в JS должно быть указано в CamelCase по аналогии с методами БЛ.

```

1  # В декоратор передаем имя функции и идентификатор нашего контекста
2  @extension( 'CamelCaseName', MY_CONTEXT_UNIQUE_IDENTIFIER )
3  # можно использовать annotations для улучшения автодополнения в PyCharm
4  def __pythonFunc( params: 'MyContextParamsModule.Class', *args ):
5      return someResult

```

Выполнение

JS Executor выполняется следующим образом:

```

1  from ExecutorExtender import ExecutorExtender
2  params = MyContextParams()
3  with
4      ExecutorExtender.getTask(MY_CONTEXT_UNIQUE_IDENTIFIER).getExecutor() as
        executor:
5          result = executor.try_exec( params, id, name, code, code_uuid,
            **kwargs )
6          if executor.hasError():
7              raise Warning( executor.getError() )

```

Пример вызова JS Executor для класса ["Расчет зарплаты"](#):

```

1  self.__executor.try_exec(params, version, 'f_' + name.replace(' ',
    '_'), code, code_uuid,
2                                     Расчет=js_record or calc,
    Сотрудник=params.man.record)

```

Благодаря конструкции **kwargs создается словарь, в котором содержатся именованные аргументы ("Расчет" и "Сотрудник" в примере выше), переданные функции при её вызове.

В дальнейшем, эти аргументы можно использовать при создании новой функции, например:

```

1  function (Расчет, Сотрудник) {
2      Расчет.Сумма = СуммаЗаМесяц() * Расчет.Процент;
3  }()

```

Кэширование

Если в функции расширения требуется сохранить какую-либо информацию для использования при следующем вызове или как результат выполнения, например, создание нового расчета при расчете зарплаты, то предлагается эти данные сохранять в экземпляре класса с параметрами контекста (задачи).

Для этого в классе BaseTaskParams описана функция getattr. Она аналогична python-функции getattr, но default принимает не значение по умолчанию, а функцию Конструктор значения по умолчанию. Это нужно для ленивой инициализации параметров.

Модули, их зависимости, репозитории SVN/GIT

Более подробную информацию можно посмотреть [здесь](#).